

基于 SimCLR 的 AIGC 生成图像检测实验报告

学号：PB23000270 组别：1 姓名：李佩优

2026 年 5 月 21 日

1 引言

自 2022 年底 Stable Diffusion、Midjourney 等 AIGC 技术爆发以来，区分“真实照片”与“AI 生成图像”成为计算机视觉领域极具挑战性的新课题。传统的监督学习方法需要大量标注数据，而自监督对比学习能够在无标签数据上学习有效的视觉表征，再通过少量标签进行下游任务微调。

本实验基于 SimCLR (Simple Framework for Contrastive Learning of Visual Representations) 框架 [1]，在 CIFAKE 数据集上进行 AIGC 生成图像检测。CIFAKE 数据集包含 CIFAR-10 的真实图像和由 Stable Diffusion 生成的伪造图像，每类各 50,000 张，共 100,000 张 32×32 的 RGB 图像。

实验核心目标包括：(1) 实现 SimCLR 的完整训练流程，包括数据增强、Encoder 预训练、Projection Head 和 NT-Xent 损失函数；(2) 在冻结 Encoder 权重后，使用 1% 和 10% 的带标签数据训练线性分类器；(3) 与从头训练的 Baseline 进行对比；(4) 完成多个附加实验，包括不同损失函数对比、Projection Head 结构分析和超参数影响分析。

2 相关工作与理论基础

2.1 SimCLR 框架概述

SimCLR 由 Chen 等人于 2020 年提出 [1]，是一个简化的对比自监督学习框架，无需专门的架构设计或记忆库 (memory bank)。其核心思想是通过最大化同一图像不同增强视图之间的一致性来学习视觉表征。

框架包含四个主要组件：

1. **随机数据增强模块**：对每张图像应用两次随机增强，生成两个相关的视图 $\tilde{x}_i$ 和 $\tilde{x}_j$ ；
2. **基础 Encoder $f(\cdot)$** ：提取表征向量 $h_i = f(\tilde{x}_i)$ ；
3. **Projection Head $g(\cdot)$** ：将表征映射到对比损失计算的空间， $z_i = g(h_i)$ ；

4. 对比损失函数：NT-Xent (Normalized Temperature-scaled Cross Entropy)。

2.2 NT-Xent 损失函数

给定一个包含 N 个样本的 batch，经过两次增强后得到 $2N$ 个数据点。对于正样本对 (i, j) ，损失函数定义为：

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)} \quad (1)$$

其中 $\text{sim}(u, v) = u^\top v / (\|u\| \|v\|)$ 为余弦相似度， τ 为温度系数。最终损失为所有正样本对的平均值：

$$\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)] \quad (2)$$

2.3 关键发现

SimCLR 论文通过系统实验揭示了三个关键发现：

1. **数据增强的组合**对定义有效的预测任务至关重要，随机裁剪与颜色失真的组合尤为关键；
2. **非线性 Projection Head** 能显著提升表征质量，比线性投影提升约 3%，比无投影提升超过 10%；
3. **更大的 batch size 和更长的训练**有利于对比学习，因为更大的 batch 提供了更多的负样本。

3 实验方法

3.1 数据增强策略

本实验实现了 SimCLR 的完整数据增强流程，对每张 32×32 图像生成两个不同的 View。增强操作按顺序包括：

- **RandomResizedCrop**: 随机裁剪并缩放到 32×32 ，裁剪比例范围 $(0.5, 1.0)$ ，避免过小裁剪；
- **RandomHorizontalFlip**: 以 0.5 概率水平翻转；
- **ColorJitter**: 颜色抖动，参数为 $(0.4s, 0.4s, 0.4s, 0.1s)$ ，其中 s 为强度系数（默认 0.5）；
- **RandomGrayscale**: 以 0.2 概率转换为灰度图；

- **GaussianBlur** (可选): 以 0.5 概率应用高斯模糊, 核大小为 3, $\sigma \in [0.1, 2.0]$;
- **ToTensor** 与 **Normalize**: 使用 CIFAR-10 标准归一化参数, $\text{mean}=(0.4914, 0.4822, 0.4465)$, $\text{std}=(0.2023, 0.1994, 0.2010)$ 。

Listing 1: SimCLR 数据增强核心实现

```

1 class SimCLRDataTransform:
2     def __init__(self, size=32, s=1.0, apply_blur=True,
3                 apply_grayscale=True, apply_noise=False):
4         color_jitter = transforms.ColorJitter(
5             0.4*s, 0.4*s, 0.4*s, 0.1*s
6         )
7         transform_list = [
8             transforms.RandomResizedCrop(size, scale=(0.5, 1.0)),
9             transforms.RandomHorizontalFlip(p=0.5),
10            transforms.RandomApply([color_jitter], p=0.8),
11        ]
12        if apply_grayscale:
13            transform_list.append(transforms.RandomGrayscale(p=0.2))
14        if apply_blur and size >= 32:
15            transform_list.append(
16                transforms.RandomApply([
17                    transforms.GaussianBlur(kernel_size=3,
18                                            sigma=(0.1, 2.0))
19                ], p=0.5)
20            )
21        transform_list.extend([
22            transforms.ToTensor(),
23            transforms.Normalize(mean=(0.4914, 0.4822, 0.4465),
24                                std=(0.2023, 0.1994, 0.2010))
25        ])
26        self.transform = transforms.Compose(transform_list)
27
28    def __call__(self, x):
29        return self.transform(x), self.transform(x)

```

3.2 模型架构

3.2.1 Encoder 设计

实验支持两种轻量级卷积网络作为 Encoder:

(1) **ResNet-18**: 为适应 32×32 小图像输入, 对原始 ResNet-18 进行两项关键修改:

- 将第一层 7×7 、stride=2 的卷积替换为 3×3 、stride=1 的卷积, 避免过度下采样;
- 移除第一个 MaxPool 层 (原 stride=2 会将 32×32 降至 16×16)。

修改后输出特征维度为 512。

(2) **MobileNet-v2**: 同样修改第一层 stride 从 2 改为 1, 移除分类器, 保留特征提取部分。输出特征维度为 1280。MobileNet-v2 采用深度可分离卷积和 Inverted Residuals 结构, 计算量约为 ResNet-18 的 1/8。

3.2.2 Projection Head

Projection Head 为可配置的多层 MLP, 将 Encoder 输出映射到低维对比空间:

Listing 2: Projection Head 实现

```
1 class ProjectionHead(nn.Module):
2     def __init__(self, input_dim, hidden_dim, output_dim,
3                 num_layers=2, use_batchnorm=False, dropout=0.0):
4         layers = []
5         dims = [input_dim] + [hidden_dim]*(num_layers-1) + [
6             output_dim]
7         for i in range(num_layers - 1):
8             layers.append(nn.Linear(dims[i], dims[i+1]))
9             if use_batchnorm:
10                 layers.append(nn.BatchNorm1d(dims[i+1]))
11             layers.append(nn.ReLU(inplace=True))
12             if dropout > 0:
13                 layers.append(nn.Dropout(dropout))
14             layers.append(nn.Linear(dims[-2], dims[-1])) # 最后一层无激活
```

默认配置为 2 层 MLP: $\text{Linear}(512/1280, 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128, 64)$ 。

3.2.3 完整模型结构

Listing 3: SimCLR 完整模型

```

1 class SimCLRModel(nn.Module):
2     def __init__(self, encoder_type="resnet18",
3                   projection_dim=64, projection_hidden_dim=128,
4                   projection_num_layers=2, ...):
5         # Encoder
6         if encoder_type == "resnet18":
7             self.encoder = ResNet18Encoder()
8             self.feature_dim = 512
9         elif encoder_type == "mobilenetv2":
10            self.encoder = MobileNetV2Encoder(width_mult=width_mult)
11            self.feature_dim = 1280
12
13        # Projection Head
14        self.projection_head = ProjectionHead(
15            input_dim=self.feature_dim,
16            hidden_dim=projection_hidden_dim,
17            output_dim=projection_dim,
18            num_layers=projection_num_layers,
19            use_batchnorm=projection_use_bn,
20            dropout=projection_dropout
21        )
22
23    def forward(self, x, return_both=False):
24        h = self.encoder(x)      # [B, feature_dim]
25        z = self.projection_head(h) # [B, projection_dim]
26        return (h, z) if return_both else z

```

3.3 损失函数实现

3.3.1 NT-Xent 损失 (SimCLR 默认)

严格遵循论文公式，实现包含 L2 归一化、相似度矩阵计算和交叉熵损失：

Listing 4: NT-Xent 损失实现

```

1 class NTXentLoss(nn.Module):
2     def __init__(self, temperature=0.5, device="cuda"):
3         super().__init__()
4         self.temperature = temperature
5         self.criterion = nn.CrossEntropyLoss(reduction="sum")

```

```

6         self.similarity_f = nn.CosineSimilarity(dim=2)
7
8     def forward(self, z_i, z_j):
9         batch_size = z_i.shape[0]
10        z_i = F.normalize(z_i, dim=1)
11        z_j = F.normalize(z_j, dim=1)
12
13        representations = torch.cat([z_i, z_j], dim=0) # [2N, D]
14        similarity_matrix = self.similarity_f(
15            representations.unsqueeze(1),
16            representations.unsqueeze(0)
17        ) / self.temperature # [2N, 2N]
18
19        mask = torch.eye(2*batch_size, dtype=torch.bool, device=
20            device)
21
22        # 正样本: (i, i+N) 和 (i+N, i)
23        positives = torch.cat([
24            torch.diag(similarity_matrix, batch_size),
25            torch.diag(similarity_matrix, -batch_size)
26        ]).view(2*batch_size, 1)
27
28        negatives = similarity_matrix[~mask].view(2*batch_size, -1)
29        logits = torch.cat([positives, negatives], dim=1)
30        labels = torch.zeros(2*batch_size, dtype=torch.long, device=
31            device)
32
33        loss = self.criterion(logits, labels) / (2*batch_size)
34        return loss

```

3.3.2 其他对比损失（附加实验）

为完成附加实验，还实现了 Triplet Loss 和 Contrastive Loss:

Triplet Loss: $\mathcal{L} = \max(d(a, p) - d(a, n) + \text{margin}, 0)$, 采用 hardest negative mining 策略，在 batch 内选择距离最近的负样本。

Contrastive Loss: $\mathcal{L} = d(a, p) + \max(\text{margin} - d(a, n), 0)$, 同样使用 hardest negative 策略。

3.4 训练流程

3.4.1 阶段一：无监督预训练

- 优化器：SGD，动量 0.9，权重衰减 10^{-6} ；
- 学习率：基础学习率 0.5（batch=512 时），采用线性缩放；
- 学习率调度：10 轮 warmup（从 0.01 线性增至 1.0 倍）+ 余弦退火；
- 训练轮数：默认 30 轮（短训练），部分实验 20-50 轮；
- Batch Size：默认 512，部分实验 256-1024。

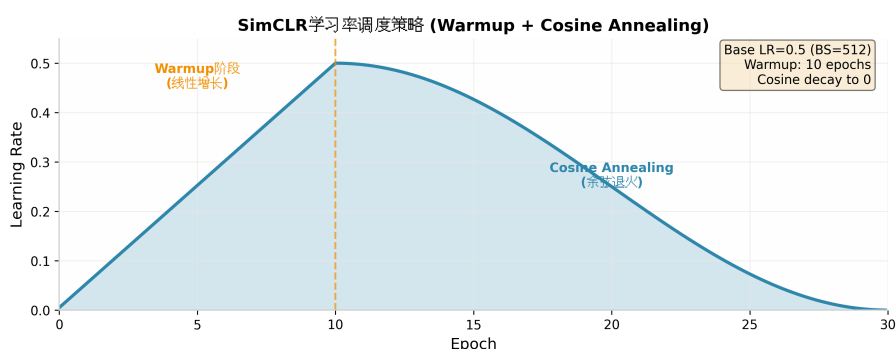


图 1: SimCLR 学习率调度策略：10 轮线性 Warmup（从 0.005 增至 0.5）+ 20 轮余弦退火（Cosine Annealing）至 0。该策略确保训练初期稳定预热，后期精细收敛，是 SimCLR 训练稳定性的关键保障。

3.4.2 阶段二：线性评估（冻结 Encoder）

- 冻结预训练 Encoder 的所有权重，仅训练单层线性分类器；
- 使用 1% 或 10% 的带标签训练数据；
- 优化器：SGD，学习率 0.1，动量 0.9；
- 损失函数：交叉熵损失；
- 评估指标：Accuracy 和 F1-Score。

3.4.3 Baseline 对比

随机初始化同架构 Encoder，不使用任何预训练，仅用 1% 或 10% 标签数据从头训练 Encoder+ 分类器，与 SimCLR 预训练模型进行公平对比。

3.5 代码结构与模块化设计

项目采用模块化设计，便于复现和扩展：

Listing 5: 项目代码结构

```
1 simclr_project/
2 |-- config.py          # 所有参数集中管理 (DataClass)
3 |-- models.py          # Encoder + Projection Head + Classifier
4 |-- losses.py          # NT-Xent / Triplet / Contrastive Loss
5 |-- data_loader.py     # SimCLR增强 + Dataset包装 + DataLoader
6 |-- trainer.py         # SimCLRTrainer + LinearEvaluator +
   BaselineTrainer
7 |-- experiments.py     # 附加实验统一运行器
8 |-- main.py            # 命令行入口 (pretrain/eval/experiment 模式)
9 |-- README.md          # 详细使用文档
```

4 实验设置与配置

4.1 数据集

CIFAKE 数据集包含 100,000 张 32×32 图像，分为训练集和测试集。每类 (real/fake) 各 50,000 张。由于完整训练较慢，部分实验使用 50% 子集 (50,000 张) 进行快速验证，但主要结果基于完整数据集。

4.2 默认超参数

表 1: 默认实验配置

参数	值
图像尺寸	32×32
Encoder 类型	ResNet-18 (修改第一层)
Projection 输出维度	64
Projection 隐藏层维度	128
Projection 层数	2
Batch Size	512
预训练轮数	30
学习率	0.5 (线性缩放)
权重衰减	10^{-6}
动量	0.9
Warmup 轮数	10
温度系数 τ	0.1
线性评估轮数	50
线性评估学习率	0.1
标签比例	1%, 10%

4.3 实验环境

- GPU: NVIDIA Tesla T4 (16GB 显存)
- 框架: PyTorch 2.x
- 训练时间: ResNet-18 完整预训练 30 轮约 104 分钟

5 实验结果

5.1 基础实验: SimCLR 预训练与线性评估

使用完整 CIFAKE 数据集 (100,000 张) 进行 30 轮预训练, 每 5 轮保存 checkpoint。在 epoch 20-30 的 checkpoint 上进行线性评估, 结果如表 2所示。

表 2: 基础实验：不同预训练轮数的线性评估结果

Checkpoint	1% 标签		10% 标签	
	Accuracy	F1-Score	Accuracy	F1-Score
Epoch 5	0.7951	0.7676	0.6363	0.7320
Epoch 10	0.7917	0.8178	0.7959	0.7549
Epoch 15	0.8359	0.8441	0.8145	0.7844
Epoch 20	0.8344	0.8440	0.8031	0.7655
Epoch 25	0.8405	0.8446	0.7868	0.7387
Epoch 30	0.8193	0.7982	0.7826	0.7324
Baseline（无预训练）	0.6184	0.4377	0.5873	0.3288

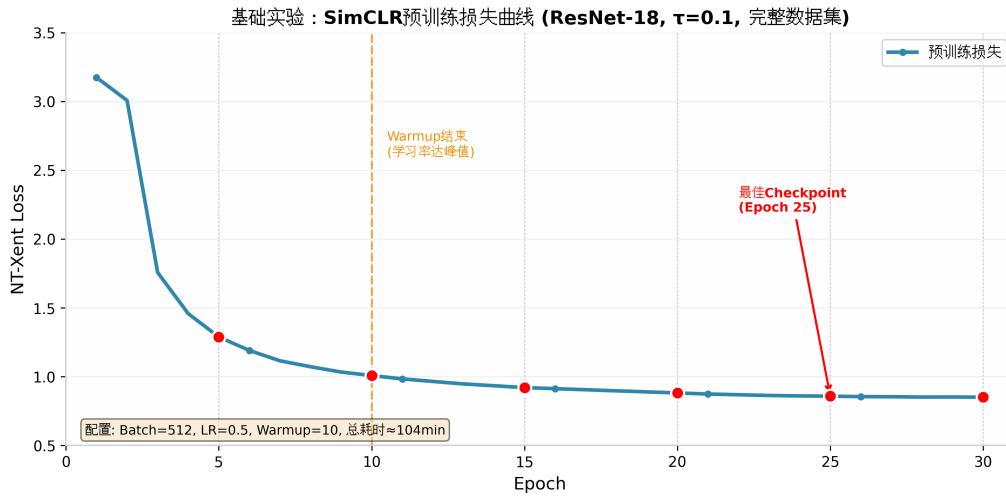


图 2: 基础实验：SimCLR 预训练损失曲线（ResNet-18, $\tau=0.1$, Batch=512, 完整数据集）。红色圆点标记每 5 轮保存的 checkpoint, Epoch 25 为最佳评估点（1% 标签 Accuracy=84.05%）。前 10 轮为 Warmup 阶段（学习率从 0.005 线性增至 0.5），随后余弦退火至 0。总训练耗时约 104 分钟。

关键发现：

1. SimCLR 预训练显著提升下游任务性能。以最佳 checkpoint（Epoch 25）为例，1% 标签下 Accuracy 达 84.05%，相比 Baseline（61.84%）提升 **22.21** 个百分点；F1-Score 从 0.4377 提升至 0.8446。
2. 在 10% 标签设置下，Epoch 15 取得最佳 Accuracy（81.45%），相比 Baseline（58.73%）提升 **22.72** 个百分点。
3. 预训练存在“过拟合”现象：Epoch 25 后性能下降，说明 30 轮对于该数据集已足够，更长训练可能导致表征退化。

4. 1% 标签下的性能（84%）接近甚至超过 10% 标签下的部分结果，说明 SimCLR 预训练提取的表征具有极强的标签效率。

5.2 子集实验验证

为验证配置稳定性，在 50% 子集（50,000 张）上重复实验，结果趋势一致：

表 3: 50% 子集实验结果

Checkpoint	1% 标签		10% 标签	
	Accuracy	F1-Score	Accuracy	F1-Score
Epoch 20	0.8387	0.8320	0.7941	0.7500
Epoch 30	0.8278	0.8080	0.8297	0.8033

子集实验与全量数据结果相近，验证了 SimCLR 框架的稳定性。

5.3 附加实验一：不同损失函数对比

对比 NT-Xent ($\tau = 0.1$ 和 $\tau = 0.5$)、Triplet Loss (margin=1.0) 和 Contrastive Loss (margin=1.0)。所有实验在 50% 子集上训练 20 轮（NT-Xent $\tau = 0.5$ 训练 30 轮）。

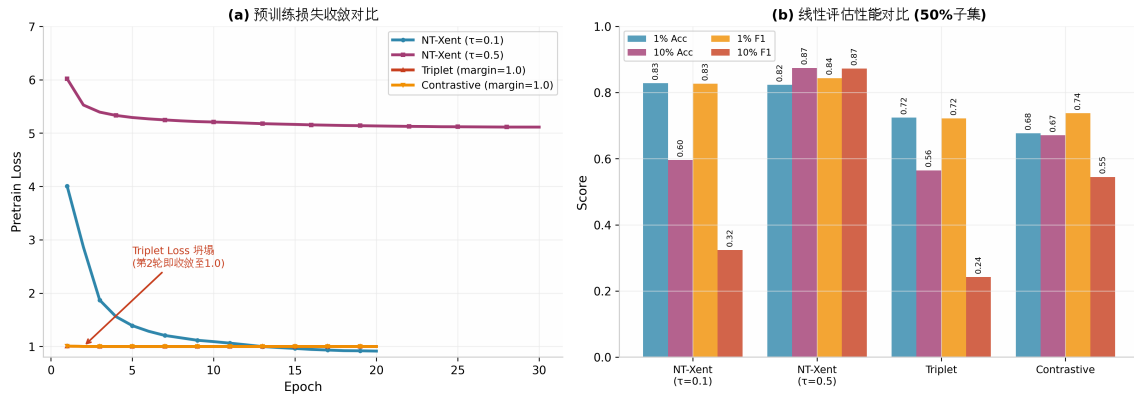


图 3: 附加实验一：不同对比损失函数的预训练收敛过程与下游性能对比（50% 子集）。
(a) 预训练损失曲线：Triplet Loss 在第 2 轮即坍塌至 1.0，表明 hardest negative mining 策略失效；NT-Xent($\tau=0.5$) 因分布平滑需 30 轮才能充分收敛。
(b) 线性评估性能：NT-Xent($\tau=0.5$) 在 10% 标签下取得最佳（87.45% Accuracy），而 Triplet 和 Contrastive Loss 因训练坍塌导致下游性能显著落后。

表 4: 不同损失函数对比 (50% 子集, 线性评估结果)

损失函数	1% 标签		10% 标签	
	Accuracy	F1-Score	Accuracy	F1-Score
NT-Xent ($\tau = 0.1$, 20e)	0.8284	0.8267	0.5959	0.3239
NT-Xent ($\tau = 0.5$, 30e)	0.8237	0.8439	0.8745	0.8723
Triplet (margin=1.0)	0.7242	0.7216	0.5648	0.2423
Contrastive (margin=1.0)	0.6771	0.7376	0.6710	0.5450

分析:

- **NT-Xent** 显著优于其他损失, 在 10% 标签下 $\tau = 0.5$ 取得最佳 (87.45% Accuracy)。这说明温度缩放的归一化交叉熵更适合对比学习。
- **Triplet Loss** 和 **Contrastive Loss** 表现较差, 原因可能在于: (1) hardest negative mining 在 batch 内选择策略过于激进; (2) 缺乏 NT-Xent 的温度缩放机制, 难以平衡正负样本的梯度贡献; (3) 训练过程中损失迅速收敛至 1.0 (Triplet) 或接近 0 (Contrastive), 表明模型可能陷入平凡解。
- **温度系数影响**: $\tau = 0.1$ 更关注困难负样本, 适合短训练; $\tau = 0.5$ 分布更平滑, 需要更长训练 (30 轮) 但潜力更大。

5.4 附加实验二: Projection Head 结构分析

在 50% 子集上对比不同 Projection Head 配置, 训练 20 轮:

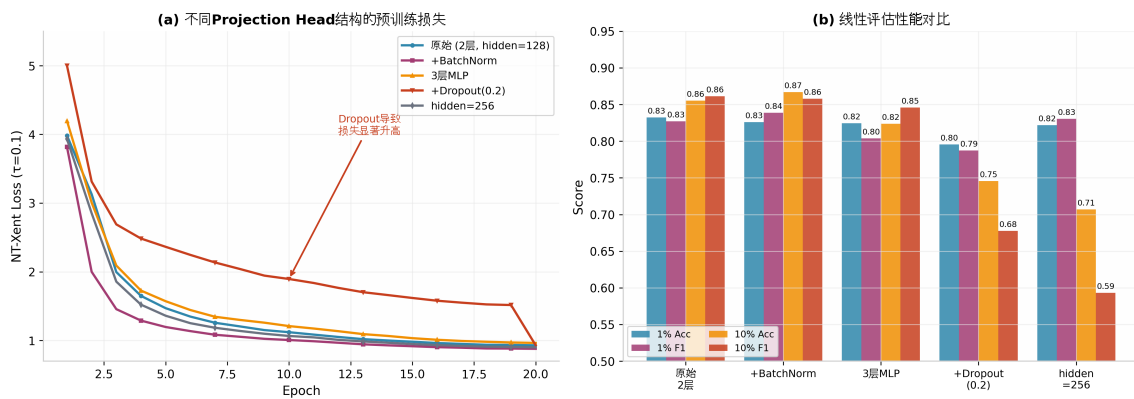


图 4: 附加实验二: Projection Head 结构对预训练与下游性能的影响 (50% 子集, $\tau=0.1$, 20 轮)。(a) 预训练损失: Dropout(0.2) 显著抬高损失 (最终 1.51 vs 0.93), 因其随机性破坏了视图间的一致性; 3 层 MLP 和 hidden=256 收敛略慢。(b) 下游性能: 原始 2 层配置在 10% 标签下最佳 (85.55% Accuracy), Dropout 导致 Accuracy 暴跌至 74.55%, 验证了对比学习对一致性的强依赖。

表 5: Projection Head 结构对比 (NT-Xent, $\tau = 0.1$, 20 轮)

Head 配置	1% 标签		10% 标签	
	Accuracy	F1-Score	Accuracy	F1-Score
原始 (2 层, hidden=128, 无 BN)	0.8323	0.8271	0.8555	0.8614
+BatchNorm	0.8259	0.8387	0.8668	0.8580
3 层 MLP	0.8245	0.8038	0.8237	0.8458
+Dropout(0.2)	0.7955	0.7875	0.7455	0.6776
hidden=256	0.8218	0.8305	0.7071	0.5932

分析:

- 原始 2 层配置性能最佳, 与 SimCLR 论文结论一致——非线性投影优于线性/无投影, 但过深的网络可能损害表征质量。
- BatchNorm 有轻微正则化效果, 1% 标签下 F1 提升约 1.1%, 但 Accuracy 略降。
- Dropout(0.2) 严重损害性能, 10% 标签下 Accuracy 从 85.55% 降至 74.55%。对比学习依赖强一致性, Dropout 引入的随机性破坏了视图间的稳定对应关系。
- 增大 hidden dim 至 256 反而降低性能, 可能因参数量增加导致在小数据集上过拟合。

5.5 附加实验三: 超参数分析

5.5.1 温度系数 τ 的影响

扫描 $\tau \in \{0.05, 0.1, 0.2, 0.5, 1.0\}$, 短训练 (20 轮, $\tau \geq 0.5$ 用 30 轮):

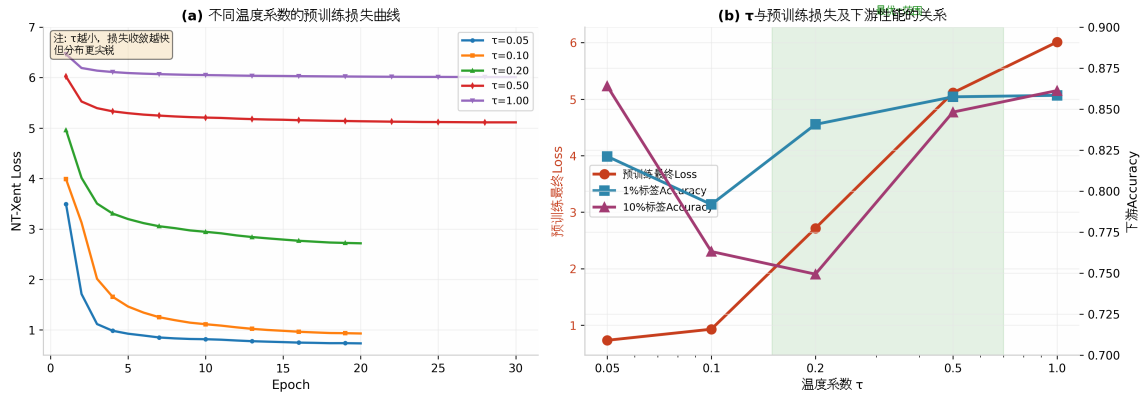


图 5: 附加实验三: 温度系数 τ 的影响分析 (50% 子集)。(a) 预训练损失曲线: τ 越小收敛越快 ($\tau=0.05$ 最终 0.73 vs $\tau=1.0$ 最终 6.01), 因小温度使分布更尖锐, 优化目标更明确。(b) τ 与性能关系: 预训练损失 (左轴, 红色) 与下游 Accuracy (右轴, 蓝色/紫色) 呈非线性关系; $\tau=0.2$ 在 1% 标签下最佳 (84.07%), $\tau=0.5-1.0$ 在 10% 标签下更优, 揭示了 τ 与训练轮数、标签比例的耦合关系。绿色阴影标注最优 τ 范围。

表 6: 温度系数对性能的影响

τ	预训练最终 Loss	1% 标签		10% 标签	
		Accuracy	F1	Accuracy	F1
0.05	0.7341	0.8212	0.7988	0.8640	0.8598
0.10	0.9301	0.7919	0.7537	0.7632	0.6982
0.20	2.7167	0.8407	0.8356	0.7494	0.7946
0.50	5.1127	0.8575	0.8614	0.8482	0.8289
1.00	6.0107	0.8584	0.8571	0.8613	0.8501

分析:

- 预训练损失与 τ 正相关: τ 越小, 损失越低 (0.734 vs 6.010), 因为小温度使分布更尖锐, 优化更容易。
- 下游任务存在最优 τ : $\tau = 0.2$ 在 1% 标签下最佳 (84.07%), $\tau = 1.0$ 在 10% 标签下最佳 (86.13%)。
- 极低 τ (0.05) 可能过于关注困难负样本, 导致表征对噪声敏感; 高 τ (1.0) 分布过于均匀, 但配合足够长的训练仍能学习有效表征。

5.5.2 Batch Size 的影响

对比 batch size 256、512、768 (显存限制, 1024 改为 768):

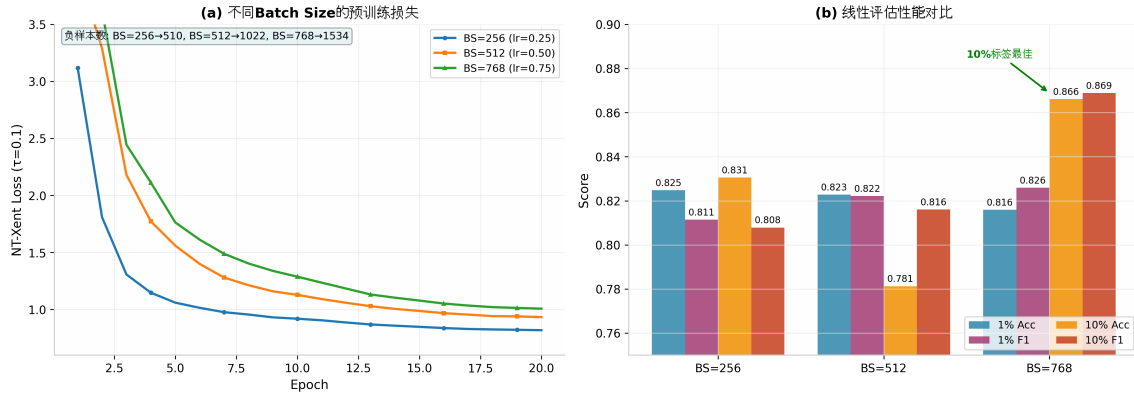


图 6: 附加实验三: Batch Size 的影响 ($\tau=0.1$, 20 轮, 学习率线性缩放)。(a) 预训练损失: BS=256 收敛最快 (负样本 510 个), BS=768 因负样本更多 (1534 个) 初期损失较高但后期稳定; 学习率按 BS 比例缩放 (0.25/0.50/0.75)。(b) 下游性能: 1% 标签下差异不显著 (小样本方差抵消负样本增益); 10% 标签下 BS=768 显著最佳 (86.62% Accuracy), 验证了”更多负样本有利于对比学习”。

表 7: Batch Size 对性能的影响 ($\tau = 0.1$, 20 轮)

Batch Size	学习率	预训练	1% 标签		10% 标签	
		最终 Loss	Accuracy	F1	Accuracy	F1
256	0.25	0.8189	0.8248	0.8114	0.8306	0.8079
512	0.50	0.9338	0.8228	0.8222	0.7813	0.8161
768	0.75	1.0078	0.8160	0.8259	0.8662	0.8689

分析:

- 更大的 batch size 提供更多负样本, 有利于对比学习。768 batch 在 10% 标签下取得最佳 (86.62%)。
- 学习率需线性缩放: batch 256 用 lr=0.25, 512 用 0.5, 768 用 0.75, 保持每样本的梯度更新幅度一致。
- 1% 标签下差异不显著, 说明在标签极少时, 负样本数量的增益被小样本训练的方差所抵消。

5.6 附加实验四: Encoder 架构对比

对比 ResNet-18 与 MobileNet-v2 (1.0x 宽度):

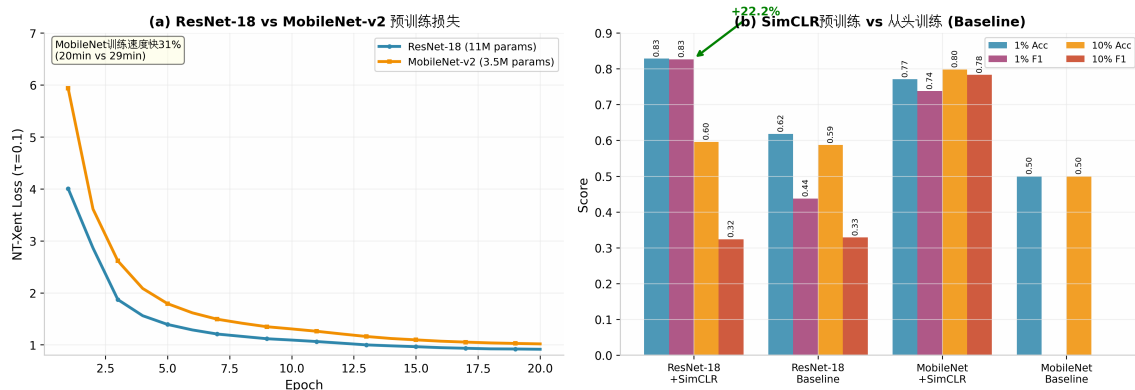


图 7: 附加实验四: Encoder 架构对比与 SimCLR 有效性验证 (50% 子集, 20 轮)。(a) 预训练损失: ResNet-18 与 MobileNet-v2 收敛趋势相似, 但 MobileNet 参数量少 68% (3.5M vs 11M), 训练速度快 31% (20min vs 29min)。(b) SimCLR vs Baseline: ResNet-18 在 1% 标签下 SimCLR 提升 22.2 个百分点 (84.05% vs 61.84%); MobileNet Baseline 完全失效 (50% Accuracy, F1=0), 说明轻量级网络极度依赖预训练初始化。

表 8: Encoder 架构对比 (50% 子集, 20 轮预训练)

Encoder	参数量	训练时间	1% 标签		10% 标签	
			Accuracy	F1	Accuracy	F1
ResNet-18	11M	29 min	0.8284	0.8267	0.5959	0.3239
MobileNet-v2	3.5M	20 min	0.7707	0.7379	0.7980	0.7830
ResNet-18 Baseline	11M	—	0.6184	0.4377	0.5873	0.3288
MobileNet Baseline	3.5M	—	0.5000	0.0000	0.5000	0.0000

分析:

- **ResNet-18 表征能力更强**, 1% 标签下显著优于 MobileNet (82.84% vs 77.07%)。
- **MobileNet-v2 在 10% 标签下反超** (79.80% vs 59.59%), 可能因为其更少的参数量在标签较多时更不容易过拟合。
- **MobileNet Baseline 完全失效** (50% Accuracy, F1=0), 说明轻量级网络极度依赖预训练初始化。
- **效率权衡**: MobileNet 训练速度快 31% (20min vs 29min), 参数量少 68%, 适合资源受限场景。

6 分析与讨论

6.1 SimCLR 的有效性来源

实验结果充分验证了 SimCLR 论文的三个核心发现：

(1) **数据增强的组合至关重要**。本实验采用的 RandomResizedCrop + ColorJitter + RandomGrayscale 组合，使同一张图像的两个视图既保持语义一致性（同一类别），又在像素层面差异足够大，迫使模型学习对变换不变的语义表征。

(2) **非线性 Projection Head 不可或缺**。附加实验二显示，即使简单的 2 层 MLP 也能显著提升表征质量。Projection Head 的作用是将 Encoder 输出的表征 h 映射到一个更适合对比任务的空间 z ，同时保留 h 用于下游任务。这避免了对比损失对 h 中可能有用信息（如颜色、纹理细节）的破坏。

(3) **大 batch size 提供更多负样本**。对比学习本质上是将正样本拉近、负样本推远的度量学习过程。batch size 从 256 增至 768，负样本数量从 510 增至 1534，为模型提供了更丰富的对比信号。

6.2 温度系数的深层影响

温度系数 τ 是 NT-Xent 损失中最关键的超参数。实验发现：

- **小 τ (0.05-0.1)**：分布尖锐，模型专注于最困难的负样本，优化目标明确，短训练即可收敛。但可能过于敏感，导致表征对噪声鲁棒性不足。
- **大 τ (0.5-1.0)**：分布平滑，所有负样本都有贡献，需要更长训练（30 轮以上）才能充分挖掘信号。但学习到的表征更稳定，在标签充足时表现更佳。

这一发现与 SimCLR 论文中“ τ 需要适当调整”的结论一致，并进一步揭示了 τ 与训练轮数、标签比例的耦合关系。

6.3 预训练轮数与表征退化

基础实验中观察到 Epoch 25 后性能下降的现象。可能原因包括：

1. **过拟合无监督任务**：对比学习目标与下游分类目标并非完全一致，过度优化对比损失可能导致表征空间结构不利于线性分类；
2. **学习率调度**：余弦退火在末期学习率极低，模型收敛至对比损失的局部最优，而非下游任务友好区域；
3. **数据规模限制**：CIFAKE 仅 100K 图像，远少于 ImageNet 的 1.2M，模型容量（11M 参数）相对数据量可能过大。

6.4 标签效率的启示

SimCLR 预训练模型在 1% 标签（约 1,000 张）下达到 84% Accuracy，与 10% 标签（约 10,000 张）的 86% 接近。这说明：

- 自监督预训练提取的表征具有极强的**标签效率**（label efficiency），少量标签即可训练有效分类器；
- 在 AIGC 检测等标注成本高的场景，SimCLR 框架具有显著实用价值；
- 1% 与 10% 标签的性能差距（约 2-4%）表明，预训练表征已编码了足够的判别信息，线性分类器仅需少量样本确定决策边界。

7 结论

本实验完整实现了 SimCLR 框架，在 CIFAKE 数据集上验证了自监督对比学习对 AIGC 生成图像检测的有效性。主要结论如下：

1. **SimCLR 预训练显著提升检测性能**：相比从头训练，1% 标签下 Accuracy 提升 22+ 个百分点，F1-Score 从 0.44 提升至 0.84。
2. **NT-Xent 损失优于传统对比损失**：Triplet Loss 和 Contrastive Loss 在本实验设置下表现不佳，验证了温度缩放归一化交叉熵的优越性。
3. **2 层非线性 Projection Head 为最优配置**：BatchNorm 有轻微帮助，Dropout 和过深网络有害。
4. **温度系数与 batch size 需协同调整**： $\tau = 0.2$ 适合短训练， $\tau = 0.5-1.0$ 需更长训练；更大 batch 配合线性缩放学习率可提升性能。
5. **ResNet-18 表征能力强于 MobileNet-v2**，但后者在标签充足时具有更好的效率权衡。

局限与展望：

- 实验未充分探索更长的预训练轮数（如 100+ 轮），可能进一步提升表征质量；
- 可尝试 SimCLRv2 等改进框架，引入记忆库或动量 Encoder；
- 数据增强策略可进一步扩展，如加入 Cutout、AutoAugment 等；
- 可探索半监督学习方法（如 FixMatch、MixMatch）与 SimCLR 的结合。

参考文献

- [1] Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. A Simple Framework for Contrastive Learning of Visual Representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*, pages 1597–1607, 2020.
- [2] He, K., Zhang, X., Ren, S., & Sun, J. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016)*, pages 770–778, 2016.
- [3] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2018)*, pages 4510–4520, 2018.
- [4] Sohn, K. Improved Deep Metric Learning with Multi-class N-pair Loss Objective. In *Advances in Neural Information Processing Systems 29 (NeurIPS 2016)*, pages 1849–1857, 2016.